

`pip install textual-debugger`
tdb, a pure-Python TUI debugger

Al Danial

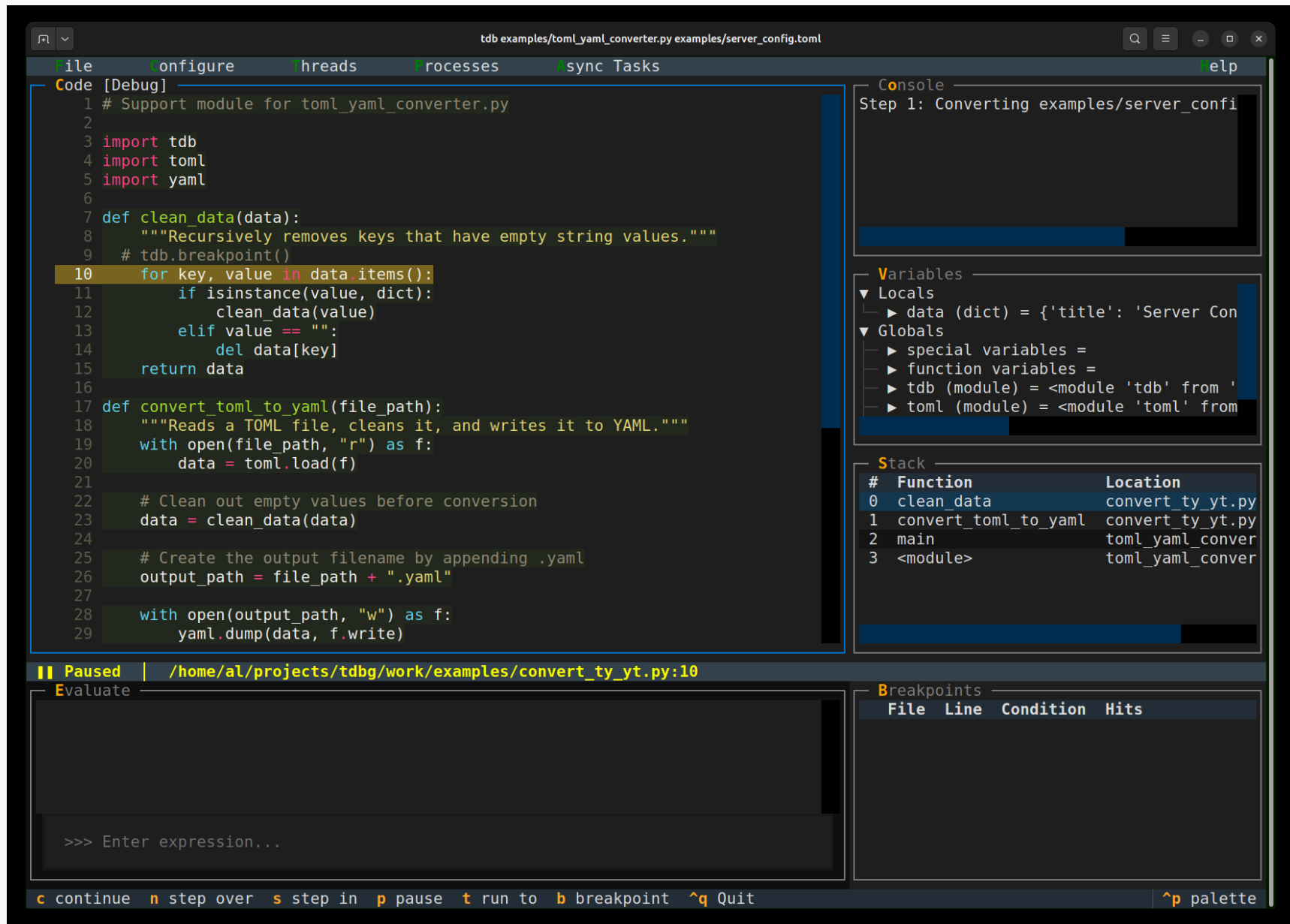
2026-05-14

PyCon US 2026 · Lightning Talks

`pip install textual-debugger`

- package `textual-debugger` provides module and command line tool **tdb**
- **Pure Python** text user interface (TUI) debugger
- Built on **textual** and **debugpy**
 - has most of the capabilities of Microsoft VS Code Python Debugger extension
 - can do several things VS Code Python Debugger cannot
- Can be **100% keyboard driven** — works over SSH, in tmux, anywhere a terminal does (mouse works too if available)

What it looks like



tdb provides the expected debugger experience

- tdb provides the expected:
 - step through code
 - inspect (and modify) variables
 - evaluate expressions in scope
 - move up and down the call stack
 - set conditional breakpoints with optional hit counts
- Examine a **post-mortem traceback** after a crash
- Get a better breakpoint() — drop in **tdb.breakpoint()** for the full TUI
- **Attach** to remotely running code

tdb provides bonus features: inspect tasks, threads, and processes

While the program is paused, peek into:

- **asyncio tasks** — with a task dependency tree view (technically a DAG, not tree)
- **threads** — every live `threading.Thread`
- **processes** — children spawned by `concurrent.futures` or `multiprocessing`

Switch focus to any of them, walk their frames, evaluate expressions in their scope.

More bonus features: server mode, headless mode, separate terminal for the debuggee

- Debug via **JSON-RPC over HTTP** in server mode — perfect for agentic debugging
- Run **headless** — no TUI, just commands and responses (for CI / scripts)
- Run debuggee in a **separate terminal** — avoids interference with tdb itself

```
pip install textual-debugger
```

```
tdb my_program.py
```

- or without installing:

```
uvx --from textual-debugger tdb my_program.py
```

- `tdb --doc`
- `tdb examples/least_squares.py`
 - basic stepping, variable inspection, expression evaluation
 - View focus, Alt v Ctrl key leaders
- `tdb examples/task_tree_v2.py`
 - c to run, immediately followed by p to pause then inspect the task tree
- `tdb examples/asyncio_deadlock.py`
 - c to run, hangs, p to pause, inspect tasks and their dependencies, detect deadlock
- `tdb examples/threading_queue.py`
 - b lines 19 and 29, c to run, inspect threads and their frames
- `tdb examples/multiprocessing_pool.py`
 - b 19, c. On breakpoint, inspect processes – wait to load! and their frames
- `tdb --terminal xterm calculator.py`